

# From Pips-Based Catan Heuristics to Markov Prediction and Expected-Hand Feasibility Estimation

**Author: Anthony van Tilburg**

**Date: April 2026**

## Abstract

This paper describes the development of a Catan decision engine for repeated simulation and automated play. The project began with simple production heuristics, especially `max_pips()` and `max_pips_and_port()`, which rank settlement locations by dice-production strength and port advantage. We then moved to a Markov-chain-inspired approach after finding Lauren Nagel's work on analyzing *The Settlers of Catan* with transition matrices. We extended the Markov direction by supporting custom/random playboards, dynamic production profiles, and bank/port trading once a player obtains trade advantages. We also incorporated PlayerOne's Board Game Analysis work on the possible ways to win at Catan as the basis for a victory-path evaluator. In attempting to make the adaptive expanded evaluator work in lightweight mode, we introduced an expected-hand feasibility formula. This formula was initially used to validate Markov timing estimates, but it may become a fast standalone predictor. Because our goal includes many simulations and repeated runs, speed is essential. The central question is whether expected-hand feasibility, enhanced with dice-probability risk adjustment and calibrated against exact PLAY outcomes, can replace or outperform the current light Markov approach.

## 1. Introduction

Catan is difficult to automate because good play depends on both short-term resource availability and long-term strategic planning. A player must decide where to place settlements, when to build roads, whether to expand, when to upgrade to cities, when to buy development cards, and how to use ports and bank trades. These decisions are shaped by dice probabilities, board layout, current hand, opponent actions, and multiple possible paths to victory.

Our development path did not begin with a general game-search algorithm. It began with practical heuristics:

`max_pips()`  
`max_pips_and_port()`

These functions evaluate settlement locations by resource-production strength and, later, by port/trading advantage. They are fast, explainable, and useful for initial placement. However, pips-based methods do not directly answer a more dynamic question:

Given my current hand and production profile,  
how many turns until I can build a city, settlement, road, or development card?

To answer that, we moved toward Markov-style resource prediction.

## 2. Starting Point: `max_pips()` and `max_pips_and_port()`

The earliest strategy logic was based on pips. In Catan, the probability of a number being rolled by two six-sided dice is not uniform. The numbers 6 and 8 are strongest; 2 and 12 are weakest. Therefore, a settlement location touching 6/8 tiles is generally more valuable than one touching 2/12 tiles.

The first heuristic, `max_pips()`, effectively asks:

Which intersection gives the highest production probability?

The second heuristic, `max_pips_and_port()`, adds trade logic:

Which intersection gives strong production and/or useful port access?

This was a natural first step. It produced explainable placement choices and avoided unnecessary complexity. But it had two limitations:

1. It evaluated production potential, not timing to future actions.
2. It did not fully model current hand, future resource accumulation, or build costs.

That led to the Markov direction.

---

### 3. Markov-Chain Direction and Lauren Nagel's Work

Lauren Nagel's TCU project, *Analysis of "The Settlers of Catan" Using Markov Chains*, uses transition matrices to analyze Catan resource states and starting-position optimality. The repository summary describes transition matrices as representing current states and probabilities of moving to proceeding states, and applies this framework to the question of optimal starting positions in Catan.

This matched our next goal: move from static pips to dynamic timing. Instead of asking only:

Which intersection has the most pips?

we wanted to ask:

Starting from this hand and this production profile,  
how long until this player can afford a target action?

The Markov approach is attractive because it can model the uncertainty of dice outcomes. In principle, it does not only track an average resource count. It can track a distribution over possible resource hands.

---

## 4. Our Markov Enhancements

We extended the Markov direction in several practical ways.

### 4.1 Custom and Random Playboards

Rather than assuming a fixed board, the engine reads a saved/custom playboard and reconstructs:

tiles  
terrain types  
dice values  
ports  
roads  
intersections  
vertex-to-roll production mappings

This allows the evaluator to work on different boards, not only the standard or a single hardcoded board.

### 4.2 Bank and Port Trading

We added bank and port-aware trading. This means the evaluator can account for:

4:1 bank trades  
3:1 generic ports  
2:1 resource-specific ports

This matters because a player with strong production of one resource and a relevant port may be able to convert surplus into missing resources faster than raw production would suggest.

### 4.3 Settlement Variants

We split settlement planning into variants:

settlement\_Or  
settlement\_1r  
settlement\_2r

because a settlement may require zero, one, or two roads first. This matters for both cost and timing.

### 4.4 Multiple Target Types

The evaluator compares target actions such as:

|                                                    |
|----------------------------------------------------|
| new settlement<br>city upgrade<br>development card |
|----------------------------------------------------|

and stores expected viable options, not just a single headline strategy.

---

## 5. PlayerOne and the Ways to Win at Catan

In parallel, we needed a way to evaluate whether an action is strategically useful, not only whether it becomes affordable soon.

For this, we used PlayerOne's Board Game Analysis work on the ways to win at Catan. The original article began as "The 102 ways to win at Catan," but the analysis was later updated; the site notes that the full list was adjusted to 143 victories after additional possibilities were found. A later summary page lists the Catan article sequence, including the 102 ways, the 143 ways, and the fastest-way analysis.

PlayerOne's Part II describes calculating expected cost of possible victories based on card-drawing probability, and Part III describes studying the fastest ways to win by looking at settlements and resource income.

Our implementation currently uses a 142-row victory-path resource file. Conceptually, this is directly inspired by PlayerOne's work: enumerate possible victory paths, estimate what each path costs, and use those paths to rank actions. In our engine, this appears at PLAY time: expected viable actions from JUMP are ranked by the 142-way evaluator, then checked by the exact guard before execution.

---

## 6. Adaptive Expanded Evaluator

We attempted to enhance the Markov system with an adaptive expanded evaluator.

The intended architecture was:

|                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------|
| light mode:<br>fast enough for regular JUMP predictions                                                              |
| heavy mode:<br><br>more complete / expanded evaluation when the light result is ambiguous or strategically important |

In practice, the lightweight path works. The heavyweight path became reachable after fixing the refinement policy, but it was too slow for normal synchronous use. The delay occurred inside the heavy event-time evaluator and trading-layer computations. Therefore, the current stable setting disables heavy refinement:

|                                                                             |
|-----------------------------------------------------------------------------|
| MARKOV_ENABLE_HEAVY_REFINEMENT = False<br>MARKOV_USE_ADAPTIVE_HEAVY = False |
|-----------------------------------------------------------------------------|

The latest diagnostics confirm that the active evaluator is light:

|                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------|
| MARKOV_ENABLE_HEAVY_REFINEMENT: False<br>MARKOV_USE_ADAPTIVE_HEAVY: False<br>used_heavy: False<br>source_mode: light<br>mode: light_reused |
|--------------------------------------------------------------------------------------------------------------------------------------------|

So at this stage, we are not benefiting from the heavy adaptive evaluator. We are benefiting from the light evaluator plus expected-hand feasibility and 142-way PLAY ranking.

---

## 7. Expected-Hand Feasibility Formula

While working on the adaptive expanded evaluator, we introduced the expected-hand feasibility formula.

Originally, it was not intended to replace Markov. It was used to check whether a Markov/FAST timing estimate was realistic.

The formula calculates:

```
expected_hand(resource, t)
=
current_hand(resource)
+
expected_production(resource, t)
```

A simplified form is:

```
expected_added_cards_i(t)
=
production_pips_i × t / 9
```

Then:

```
expected_hand_i(t)
=
current_hand_i + expected_added_cards_i(t)
```

For example, a log line may show:

```
hand=[1, 1, 1, 0, 0]
pips=[5, 12, 7, 0, 8]
rounds=3.05
add=[1.69, 4.07, 2.37, 0, 2.71]
exp=[2.69, 5.07, 3.37, 0, 2.71]
need=[2, 3, 0, 0, 0]
short=[0, 0, 0, 0, 0]
```

Here, exp=[...] is produced by the expected-hand feasibility formula. It is not the direct output of the Markov state distribution. The Markov/FAST layer proposes a candidate timing; the expected-hand layer checks whether the average expected hand at that timing can pay the cost.

This was a turning point. The expected-hand formula was originally a validator, but it can be turned into a predictor by asking:

What is the earliest future t where expected\_hand(t) can pay the action cost?

## 8. Two-Dice Probability Distribution

A useful way to understand this is to convert the two-dice probability distribution into an expected list of rolls.

There are 36 possible outcomes when rolling two six-sided dice. The expected 36-roll distribution is:

```
[2,
3, 3,
4, 4, 4,
5, 5, 5, 5,
6, 6, 6, 6, 6,
7, 7, 7, 7, 7, 7,
8, 8, 8, 8, 8,
9, 9, 9, 9,
10, 10, 10,
11, 11,
12]
```

So if a player produces brick on number 5, then over 36 rolls we expect about 4 brick cards from that tile, because 5 appears 4 times in the expected 36-roll distribution.

For 12 rolls, we can make a rough representative expected list by scaling the 36-roll distribution down:

[3, 4, 5, 6, 6, 7, 7, 8, 8, 9, 10, 11]

This is not a promise that these exact 12 rolls will happen. It is only a representative average pattern.

A real 12-roll sequence could be:

[2, 3, 5, 6, 6, 6, 7, 7, 8, 9, 10, 12]

or:

[3, 3, 4, 6, 6, 7, 8, 8, 9, 10, 12, 12]

Both are plausible. This is exactly the difference between expected value and actual outcome.

So when we say:

Expected brick after 4 turns = 1.0

we do not mean the player certainly has one brick. We mean that across many possible futures, the average amount of brick is one. In one actual game path the player may have zero brick; in another, one brick; in another, two or more.

### 9. Clarifying the “Expected Brick After 4 Turns = 1.0” Example

Suppose a player’s brick production has an aggregate probability of:

$p = 9/36 = 0.25$

This could happen if the player’s brick production is spread across numbers whose pips add to 9, such as a 6/8 plus a 5/9-equivalent combination, depending on board layout and settlements.

After 4 dice-roll opportunities:

expected brick =  $n \times p = 4 \times 0.25 = 1.0$

So the expected-hand formula says:

Expected brick after 4 turns = 1.0

But this does not mean the player definitely has one brick. It means the average over many possible futures is one brick.

If we approximate brick production as a binomial event with:

$n = 4$  rolls

$p = 0.25$  chance of brick per roll

then the probability distribution is:

| Bricks after 4 rolls | Probability |
|----------------------|-------------|
| 0                    | 31.6%       |
| 1                    | 42.2%       |
| 2                    | 21.1%       |
| 3                    | 4.7%        |
| 4                    | 0.4%        |

So even though the expected value is exactly 1.0, the player still has about a 31.6% chance of having zero brick after four rolls.

This is the central accuracy issue for expected-hand feasibility:

Expected value tells us the average.

It does not tell us the confidence.

Markov is theoretically better at this because it can model the probability distribution over hands. Expected-hand feasibility is faster and simpler, but it needs a risk or confidence adjustment.

## 10. Why Expected-Hand Feasibility May Replace Markov

The expected-hand formula is much cheaper than the Markov state-transition approach. It requires only:

current hand  
production pips  
future horizon  
build cost  
trade rates

For each candidate action, the estimator can search forward in small increments:

$t = 0.0, 0.25, 0.5, 0.75, \dots$

and find the first time where the expected hand can pay the cost.

This is far faster than building or querying expanded resource-state transition matrices. It is also easier to debug because every output can be explained:

current hand  
+ expected production  
- required cost  
+ possible trades  
= payable or not payable

Since our goal includes many simulations/runs, this speed advantage is critical.

---

## 11. Accuracy Justification Compared with Markov

Expected-hand feasibility is less probabilistically complete than Markov, because it uses averages. However, it may still be accurate enough for fast-forward prediction if we add three safeguards.

### 11.1 Safety Margins

Instead of accepting:

$\text{expected\_brick} \geq 1.0$

we can require:

$\text{expected\_brick} \geq 1.0 + \text{margin}$

For example:

$\text{expected\_brick} \geq 1.25$   
or:  
 $\text{expected\_brick} \geq 1.50$

A larger margin reduces false positives where the average says the resource is available but the probability of actually having it is still too low.

### 11.2 Confidence Scores

Every expected-hand estimate should return a confidence score:

high  
medium  
low

A prediction based on direct strong production should be higher confidence than one depending on a rare resource or multiple trades.

Example:

Need 1 brick  
Expected brick = 1.0  
Probability of zero brick  $\approx 31.6\%$   
Confidence = medium/low

### 11.3 Exact PLAY Guard

Most importantly, the prediction does not execute directly. PLAY still checks the real hand and legal board state.

So even if expected-hand feasibility is optimistic, the exact guard prevents illegal execution.  
The architecture becomes:

Expected-hand estimator:  
fast prediction

142-way evaluator:  
strategic ranking

Exact guard:  
truth check before execution

This makes expected-hand feasible as a practical engine even if it is not theoretically as pure as Markov.

---

## 12. Proposed Chapter 9 Extension: Probability-Aware Safety Margins

The dice-distribution example should be used directly in the accuracy chapter.

For a target resource with aggregate production probability  $p$ , and future horizon  $n$ , the expected count is:

$$E[X] = n \times p$$

But we also estimate:

$$P(X = 0) = (1 - p)^n$$

or more generally:

$$P(X \geq \text{required\_amount})$$

For example:

$p = 0.25$   
 $n = 4$   
required brick = 1

Then:

$P(\text{at least 1 brick})$   
=  
 $1 - P(0 \text{ bricks})$   
=  
 $1 - 0.75^4$   
=  
68.4%

So an expected value of 1.0 corresponds to only about 68.4% confidence of having at least one brick.

This suggests a practical rule:

If expected value barely equals the requirement,  
treat the estimate as low/medium confidence.

If expected value exceeds the requirement by a margin,  
treat it as higher confidence.

For example:

expected = 1.00, need = 1 → medium/low confidence  
expected = 1.50, need = 1 → medium confidence  
expected = 2.00, need = 1 → high confidence

This is not full Markov, but it is a lightweight probability-aware correction.

---

## 13. Integration with the 142-Way Victory-Path Evaluator

The expected-hand estimator becomes most useful when combined with PlayerOne-style victory paths.  
For each candidate action, we can estimate:

1. How soon can this action be afforded?
2. How does this action change future production?
3. Which victory paths become faster after this action?
4. Which action minimizes expected turns to a winning path?

Example:

City at 38:

cost = 2 wheat + 3 ore  
production gain = doubled ore/wool  
expected turns to best path = 16.2

City at 29:

cost = 2 wheat + 3 ore  
production gain = doubled wheat/ore  
expected turns to best path = 14.8

Choose city at 29.

This extends the current PLAY ranking idea into a fuller strategic planner.

---

#### 14. Proposed Validation Plan

We should not immediately replace Markov. Instead, run Markov/light and expected-hand side by side.

For each candidate action, log:

player  
action  
target  
Markov/light score  
expected-hand score  
expected-hand confidence  
exact guard result  
actual executed action

Then compare:

1. Runtime
2. Exact-guard success rate
3. Timing error
4. Ranking accuracy
5. Strategic path improvement
6. Simulation throughput

The key empirical question is:

Can expected-hand feasibility match or beat light Markov's practical prediction quality while running much faster?

---

#### 15. Current Position

The current stable settings are:

MARKOV\_ENABLE\_HEAVY\_REFINEMENT = False  
MARKOV\_USE\_ADAPTIVE\_HEAVY = False

Therefore, the current engine is using:

light Markov/FAST timing  
expected-hand feasibility correction  
142-way PLAY ranking  
exact guard  
same-turn chaining

It is not currently benefiting from the heavy adaptive expanded evaluator.



The expected-hand feasibility formula is currently one of the most promising parts of the system. It already explains and corrects much of the practical timing behavior. The next step is to make it a standalone estimator.

---

## 16. Next Steps

### Step 1: Create a Standalone Expected-Hand Estimator

Create:

```
core/resource_time_estimator.py
```

Initial functions:

```
estimate_expected_hand_after_turns(...)
compute_payability_with_trades(...)
estimate_first_payable_turn(...)
estimate_action_time(...)
rank_action_times_for_player(...)
```

### Step 2: Add Probability-Aware Confidence

Use two-dice probabilities to estimate confidence.

For a resource with aggregate production probability  $p$  over  $n$  rolls:

```
expected = n * p
P(at least one) = 1 - (1 - p)^n
```

For multi-resource costs, confidence should be lower when several independent scarce resources are needed.

### Step 3: Print Side-by-Side Scores

Example:

```
Red:
dev_card   Markov=3.05 EH=3.25 conf=medium
city@38    Markov=3.05 EH=2.75 conf=high
settlement19 Markov=4.00 EH=9999 conf=low
```

### Step 4: Compare Against Exact PLAY

Use exact PLAY guard as the truth source:

```
Predicted payable?
Actually payable?
Executed successfully?
```

### Step 5: Connect to Victory Paths

Use expected-hand estimates to evaluate:

```
turns to next action
turns to complete best victory path
production gain after candidate action
```

---

## Conclusion

This project began with simple pips-based heuristics: `max_pips()` and `max_pips_and_port()`. These methods are fast and intuitive but limited to static production evaluation. Inspired by Lauren Nagel's Markov-chain analysis of Catan, we moved toward Markov-based timing prediction and extended it to custom boards, port-aware trading, settlement variants, and multiple target types. We also integrated PlayerOne's Board Game Analysis work on Catan victory paths into a 142-way strategic evaluator.

The adaptive expanded evaluator works in lightweight mode, but heavyweight refinement is currently too slow for repeated simulation. In the process of making the light evaluator practical, we introduced the

expected-hand feasibility formula. This formula was first used as a validator for Markov predictions, but it may become the primary prediction method.

The expected-hand approach is faster, more explainable, and easier to combine with victory-path planning. Its main weakness is that it uses averages, not full probability distributions. However, this can be improved with safety margins, confidence scoring, two-dice probability calculations, and exact PLAY guards.

The next research question is:

Can probability-aware expected-hand feasibility replace light Markov as the primary engine for fast Catan simulation?

That should be tested empirically by running both predictors side by side and comparing speed, accuracy, exact-guard success, and strategic outcomes.

---

### **Prior Work and Acknowledgements**

This project builds on two important strands of prior work.

First, the Markov-chain direction was inspired by Lauren Nagel’s analysis of “The Settlers of Catan” using transition matrices. Nagel’s work applies Markov-chain reasoning to Catan resource states and starting-position evaluation, which provided a natural next step after our earlier pips-based heuristics.

Second, the victory-path component was inspired by PlayerOne’s Board Game Analysis series on the ways to win at Catan. The original article began as “The 102 ways to win at Catan,” and the analysis was later updated to 143 possible victories. This idea of enumerating victory paths directly influenced our current 142-way resource-requirements file and the PLAY-time action-ranking logic.

OpenAI’s ChatGPT was used as a writing and coding assistant during the development of this paper. It helped draft explanatory text, organize sections, compare algorithmic approaches, and refine implementation ideas. The author remains responsible for the final content, interpretation, code decisions, and validation.

---

### **Use of AI Assistance**

OpenAI’s ChatGPT was used as an assistant in preparing this paper. The assistant helped structure the paper, rewrite draft sections, explain algorithmic alternatives, and formulate pseudocode and validation plans. The system was also used during software development discussions for the Catan engine.

All claims, implementation decisions, experimental interpretations, and final wording were reviewed and selected by the author. ChatGPT was not treated as an independent source of evidence; external claims are attributed to their original sources where applicable.

---

### **References**

1. Nagel, L. (2021). “Analysis of ‘The Settlers of Catan’ Using Markov Chains”. Texas Christian University.
2. PlayerOne. (2017). “The 102 ways to win at Catan – Part I”. Board Game Analysis.  
<https://www.boardgameanalysis.com/the-143-ways-to-win-at-catan-part-i/>
3. PlayerOne. (2017). “The 143 ways to win at Catan – Part II”. Board Game Analysis.  
<https://www.boardgameanalysis.com/the-143-ways-to-win-at-catan-part-ii/>
4. PlayerOne. (2017). “The 143 ways to win at Catan – Part III”. Board Game Analysis.  
<https://www.boardgameanalysis.com/the-143-ways-to-win-at-catan-part-iii/>